

AgentForge Studio

The Deployment Agent — CI, AWS and Vercel delivery for a generated Next.js app

A thread-per-request HTTP service that takes a built Next.js application, writes it the CI/CD workflows and infrastructure it needs, stops for a human's approval, then puts it on AWS EC2, AWS ECS Fargate or Vercel — watching it until it answers, and able to stop it and delete it again.

Repository	https://github.com/Ravindu200232/full-stack-ai-web-developer
Branch	deployment-agent
Scope	deployment-agent/ and studio/components/deploy/
Written	2026-08-14

Contents

Contents	2
1. Overview	3
2. What it does	3
3. Technology and tools	7
4. How it works	10
1. 1. Mount and recover	10
2. 2. Analyze (DRAFT to ANALYZING)	10
3. 3. Plan	10
4. 4. Generate	10
5. 5. Validate and build (to REVIEW_READY)	10
6. 6. Approve (REVIEW_READY to BOOTSTRAPPING)	10
7. 7. GitHub identity and OIDC subjects	11
8. 8. Prepare the provider	11
9. 9. Hand GitHub what it needs	11
10. 10. Apply, commit, push (to CI_RUNNING)	11
11. 11. Watch the workflow (CI_RUNNING to DEPLOYING to VALIDATING)	11
12. 12. Validate live (VALIDATING to LIVE)	11
13. 13. Adopt, or retire	12
5. The code, file by file	13
6. Why it is built this way	15
7. Interfaces	17

1. Overview

The deployment agent is a copy of a standalone product (DeployForge) mounted inside AgentForge's own process, and it is deliberately kept as a copy: `deploy_agent/__init__.py` says a future re-sync should be a directory copy, not a merge. Every AgentForge-specific seam is collected in `deploy_agent/bridge.py`, and only three files in the copy import it — `config.py` (where `state.db` lives), `tools.py` (which Ollama the readiness probe asks) and `orchestrator.py` (which model writes the plan). Two things were renamed because AgentForge has a module of each name at the root of `sys.path`: `agents/` became `dfagents/`, `server.py` became `dfserver.py`.

A deployment is two halves separated by a human. The first half touches neither the user's repository nor any cloud account. `IntakeAgent` copies the project into an isolated worktree under `production-ready/.deploy/runs/<id>/worktree`; `PlannerAgent` asks a Gemma model for a schema-validated plan; `ArtifactGeneratorAgent` writes the CI workflows, the CloudFormation template (or the Vercel config), the release script and a set of bounded source patches into that copy; `SecurityValidatorAgent` checks them; and the orchestrator runs a real install and build inside the staged copy to find out whether the thing actually compiles. The run parks at `REVIEW_READY` with a readiness score and a unified diff the studio renders.

The second half re-verifies every artifact's SHA-256 against what was reviewed, applies the bootstrap CloudFormation stack (or links the Vercel project), writes the runtime secret, hands GitHub the variables and secrets its workflows read, copies the reviewed files into the real repository, commits, pushes, and then watches the GitHub Actions run it triggered. Everything that could differ per provider is a fact in a table (`providers/spec.py`: `TargetProfile`) rather than a subclass; the only genuine behavioural variance is "get the provider ready and tell me what to put into GitHub", which is what `ProviderPrep` carries.

The run itself is a real state machine enforced in SQLite: `StateStore.transition_run` raises on any edge not in `ALLOWED_TRANSITIONS`. A supervisor sweeps every 15 seconds for runs abandoned by a process restart. A great deal of the code exists because of specific, measured failures — a build that needed a database to compile, an OIDC provider a redeployment deleted out from under itself, a deployment stuck in `VALIDATING` for a day that could not be deleted — and the docstrings name them.

2. What it does

Feature	What it does	Where
Generated CI workflow with a real MongoDB service container	<code>ci.yml</code> runs on <code>pull_request</code> and on push to the detected branch, path-filtered to the service root. The single <code>validate</code> job declares a <code>mongo:7</code> service on <code>27017:27017</code> with a <code>mongosh</code> ping health check (10s interval, 5s timeout, 10 retries), installs, builds with the placeholder environment, runs a production-only dependency audit (continue-on-error, gated on <code>hashFiles</code> of the lockfile), and on the EC2 target asserts that <code>.next/standalone/server.js</code> exists.	<code>deployment-agent/deploy_agent/dfagents/generator.py</code> (<code>_ci_workflow</code> , <code>_build_services_yaml</code> , <code>_toolchain</code>)
AWS EC2 deployment workflow: S3 plus SSM Run Command	<code>deploy.yml</code> builds, packages <code>.next/standalone</code> (copying <code>.next/static</code> in, and <code>public/</code> if it exists) into <code>/tmp/app.tar.gz</code> , assumes the <code>deploy</code> role over GitHub OIDC, uploads the tarball and <code>deploy/release.sh</code> to S3 under <code>releases/<sha>/</code> , dispatches <code>AWS-RunShellScript</code> over SSM with a 600-second timeout, then polls <code>get-command-invocation</code> itself 60 times at 5s — the invocation is not queryable the instant <code>send-command</code> returns, and the built-in waiter caps out below the release timeout. It finishes by curling the stack's <code>ApplicationUrl</code> output plus the health path.	<code>deployment-agent/deploy_agent/dfagents/generator.py</code> (<code>_deploy_workflow</code>)

Feature	What it does	Where
The EC2 release script, run on the instance	Fetched from S3 per deployment rather than baked into UserData, so changing it does not require replacing the instance. It unpacks into <code>/opt/app/releases/<sha></code>, rebuilds <code>/opt/app/shared/.env</code> from Secrets Manager through a jq allow-list of exactly the keys the contract declares, swaps the current symlink atomically with <code>mv -Tf</code>, restarts the nextjs systemd unit, polls the health path 30 times at 2s, prunes to the five most recent releases so rollback has somewhere to go, and dumps <code>journalctl -u nextjs -n 50</code> on failure.	<code>deployment-agent/deploy_agent/dfagents/generator.py</code> (<code>_release_script</code>)
AWS ECS Fargate path, image built on the runner and never locally	A three-stage Dockerfile, a Fargate task definition and an ECS deploy workflow that logs into ECR, runs <code>docker build</code> with <code>--network=host</code>, tags with <code>github.sha</code> and never <code>:latest</code>, renders the task definition with an inline python3 heredoc, registers the revision, calls <code>update-service --desired-count 1</code> and then <code>aws ecs wait services-stable</code> so a stalled rollout fails the workflow instead of reporting a green deploy over the old image. <code>required_tools</code> for this target is <code>git</code>, <code>gh</code>, <code>aws</code> — Docker is deliberately absent.	<code>deployment-agent/deploy_agent/dfagents/generator.py</code> (<code>_ecs_deploy_workflow</code>, <code>_dockerfile</code>, <code>_task_definition</code>), <code>deployment-agent/deploy_agent/deployment_agent/providers/spec.py</code>
Bounded, declared source patches for deployment compatibility	Adds output: standalone to the Next config (or creates a minimal one), adds a <code>GET /api/health</code> route if none exists (App Router or Pages), rewrites an eager <code>lib/mongodb.js</code> to connect on first use, defers the Better Auth catch-all route's handler to the first request, strips an explicit <code>baseUrl</code> from the auth client so it is same-origin behind nginx, and adds <code>export const dynamic = force-dynamic</code> to pages and layouts that read the database. Every patch is recorded with a path, a reason and a change, and shows up in the reviewable diff.	<code>deployment-agent/deploy_agent/dfagents/generator.py</code> (<code>_apply_compatibility_patches</code>)
Local production build as a gate, not a suggestion	The orchestrator runs the real install and build commands inside the staged copy with <code>NODE_ENV</code> cleared and the placeholder <code>MONGODB_URI</code> set, scores 15 points for a pass and 8 for a fail, parses the npm audit summary out of the output to cap the security category, and on the AWS targets fails the build itself if <code>.next/standalone/server.js</code> was not produced. Deployment is refused unless the build category is at least 15.	<code>deployment-agent/deploy_agent/deployment_agent/orchestrator.py</code> (<code>_validate_build</code>), <code>deployment-agent/deploy_agent/dfagents/deployer.py</code> (<code>_validate_request</code>)
One bounded model-chosen repair attempt after a failed build	If the build fails, the planner is shown the last 8000 characters of redacted output and may pick only from four predefined actions (<code>ensure-standalone-output</code>, <code>ensure-type-safe-health-route</code>, <code>same-origin-auth</code>, <code>normalize-alert-variant</code>) or none. The generator applies only those, re-validates and rebuilds. The alert-variant repair scans at most 2000 files, skips anything over 512 KB, and only normalizes an unsupported Alert variant to the component default.	<code>deployment-agent/deploy_agent/dfagents/planner.py</code> (<code>repair_build</code>, <code>BUILD_REPAIR_SCHEMA</code>), <code>deployment-agent/deploy_agent/dfagents/generator.py</code> (<code>repair_compatibility</code>)
Artifact integrity re-check between review and deploy	<code>_validate_request</code> reloads the manifest, checks its schema version is exactly 3, checks the runtime strategy matches the target profile, checks the persisted artifact set equals <code>agent_owned_files</code>, and re-hashes every staged file against its recorded SHA-256. When copying into the real repository, a file that existed before must still match its recorded original hash or the run aborts with 'Source file changed after review; re-run analysis'.	<code>deployment-agent/deploy_agent/dfagents/deployer.py</code> (<code>_validate_request</code>, <code>_apply_reviewed_artifacts</code>)

Feature	What it does	Where
Push, or open a pull request when the branch is protected	Stages everything except .env value files, node_modules, .next and .vercel, force-adds .env.example, refuses outright to commit any staged file whose name begins with .env and is not .env.example, then pushes to the default branch. If that push is rejected it pushes deployment-agent/<run8> and opens a PR instead, and the run parks in CI_RUNNING with a waiting event carrying the PR URL. Git authenticates through a per-invocation gh auth git-credential helper rather than mutating the user's global config.	deployment-agent/deploy_agent/dfagents/deployer.py (_commit_and_push, github_credential_args)
In-app AWS onboarding: SSO device flow, quotas, permission simulation	Drives the IAM Identity Center device authorization flow so the operator never types a credential; the browser only sees a verification URL, a user code, account and role names, and an opaque vault reference. It compares the target's service quotas against current usage and simulates the target's required IAM actions with SimulatePrincipalPolicy, reporting unknown rather than blocked when the identity cannot introspect itself. It can also emit a CloudFormation template an administrator applies once to grant the permissions.	deployment-agent/deploy_agent/deployment_agent/aws_onboarding.py, deployment-agent/deploy_agent/deployment_agent/credentials.py
MongoDB URI preflight that names the actual problem	Connects and pings with a 6-second budget before any cloud resource is touched, and classifies the failure as auth, dns, unreachable (almost always the Atlas IP allowlist) or malformed. It rejects query options the Node driver does not know, because those pass this check but fail the app's build, and warns when the URI names no database, since everything would then land in test. It never returns the URI or any credential.	deployment-agent/deploy_agent/deployment_agent/mongodb_check.py
Live monitoring snapshot across three providers	One /monitor call collects GitHub workflow runs (and the PR, if the push became one), then either EC2 instances plus SSM release history, ECS services/tasks/deployments plus the running image, or Vercel deployments, domains and build logs — with the CloudFormation stack, its recent events and up to 100 CloudWatch lines shared by both AWS collectors. It probes / and /api/health live, recomputes readiness out of 100, and drives the transitions to VALIDATING, LIVE or FAILED.	deployment-agent/deploy_agent/dfagents/monitor.py
Cancel: stop the run and the workflow, delete nothing	Moves the run to CANCELLED, which is outside the supervisor's reconciled set so it will not be swept back to life, and best-effort cancels the live GitHub Actions run, because that is the thing still writing to the instance. AWS is left completely alone and still billing, and the UI says so in as many words. The worker thread is not stopped — it cannot be, it is inside a boto3 or git call — it finds out when its next state transition is rejected.	deployment-agent/deploy_agent/dfagents/deployer.py (cancel, _cancel_workflow_run), studio/components/deploy/DeployDanger.jsx
Teardown: irreversible, and reachable even from a stuck run	Deletes the <slug>-service and <slug>-bootstrap stacks (or the Vercel project), emptying the ECR repository and every object version and delete marker in the S3 bucket first, since CloudFormation cannot delete either while non-empty. An active run is refused without force; with force it is cancelled first and then torn down, in that order, so a teardown does not race a workflow that would recreate the stack.	deployment-agent/deploy_agent/dfagents/deployer.py (start_teardown, _teardown_worker, _drain_stack_resources)
Rollback on both providers	On EC2 an SSM script relinks /opt/app/current to the most recent release directory that is not the current one and restarts the service. On Vercel the second-newest READY deployment is promoted through the REST API. Both move the run to ROLLED_BACK, which has edges back to BOOTSTRAPPING and on to DESTROYED.	deployment-agent/deploy_agent/dfagents/deployer.py (rollback, _rollback_vercel)

Feature	What it does	Where
A supervisor that recovers runs abandoned by a process restart	A deployment is driven by a daemon thread; when the process exits mid-deployment that thread disappears while the SQLite row stays active, locking the project against every later deployment. The reconciler runs once at startup and then every 15 seconds, treats an absent in-memory project key as the orphan signal, and for a run with a repository simply re-reads the authoritative remote state through the monitor. Orphaned deployments fail after 20 minutes, orphaned analyses after 90. It never starts, retries or rolls anything back.	deployment-agent/deploy_agent/deployment_agent/supervisor.py
Evidence export: a ZIP and a PDF	The ZIP carries every reviewed artifact (re-redacted on the way in), the run record, the full event stream and any screenshots. The PDF is a ReportLab document with the readiness table, the detected services, the last 80 events, the masked remote snapshot and up to ten images. Dashboard capture through Playwright is best-effort and returns an empty list when unavailable, which under AgentForge it always is, since the dashboard it screenshotted was not part of the port.	deployment-agent/deploy_agent/deployment_agent/exporter.py
The studio Deploy tab: a stepper, ten monitor views and a console	Picks a target, shows which of GitHub, AWS-or-Vercel and MongoDB is still unconnected, blocks the button until unit tests pass or the user explicitly overrides, then renders a per-stage stepper from the agent's own event stream. After that: Overview, Pipeline, Repository, CI/CD (with the generated workflow source), Infrastructure, IAM and Security, Logs, API Validation and Evidence, plus a sticky console with Live Logs, Agent Events, Terminal, Prompt Trace and Errors channels.	studio/components/deploy/DeployPanel.jsx, studio/components/deploy/MonitorViews.jsx, studio/components/deploy/MonitorPipeline.jsx, studio/components/deploy/MonitorRepo.jsx, studio/components/deploy/MonitorSecurity.jsx, studio/components/deploy/MonitorConsole.jsx
Autopilot: one button drives analyze, review, deploy and adopt	AgentForge's server.py runs the whole sequence on a background thread, draining the agent's events into the studio log by cursor, refusing to deploy a plan written without a model, and finally copying run.json, events.json, monitor.json and link.json into the project's .agentforge/deploy/ so reopening it a week later still shows what was deployed and where. A torn-down run's record is archived to .agentforge/deploy-archive/<run_id> rather than deleted, because the record is the evidence.	server.py (_deploy_autopilot, _deploy_wait, adopt_deploy, retire_deploy)
Long-call job shim so the studio never holds a blocking request open	POST /api/deploy/jobs starts a proxied call to the agent on a thread and answers immediately with a djob_id; the studio polls GET /api/deploy/jobs/<id> at 300 ms then 900 ms. Jobs cannot start jobs, finished jobs are reaped after 900 seconds, and at most 200 are kept. This is what lets a 25-second monitor collection or a multi-minute onboarding probe run from a browser without a request timeout.	server.py (deploy_job_start, _deploy_job_run, deploy_job_poll), studio/lib/api.js (deployJob)

3. Technology and tools

Technology	Version	What it does here	Why
Python standard library (http.server.ThreadingHTTPServer)	—	The agent's whole HTTP API on 127.0.0.1:7834 — a SimpleHTTPRequestHandler subclass with hand-rolled routing and security headers on every response.	deploy_agent/mount.py states it: a ThreadingHTTPServer gets a thread per request, so the long blocking work (boto3, gh, npm ci) blocks only its own request's thread. Rewriting thirty-five handlers onto AgentForge's websocket loop would put every one of those blocking calls back on a loop — the bug the SRS agent hit with uvicorn and ReportLab.
SQLite	—	The single source of run state: runs, events (auto-increment id, indexed per run), artifacts (keyed run_id+path) and evidence, behind StateStore with an RLock and check_same_thread=False.	The event id is what replaced the websocket. GET /api/runs/{id}/events?after=N is a replayable cursor read, so a browser reload loses nothing; server.py says this is the whole reason the agent's websocket on :7835 was not ported.
AWS CloudFormation	—	Both AWS targets' infrastructure, applied as a named change set (deployment-agent-<epoch>) with CAPABILITY_NAMED_IAM and previewed as a change_set event before execution.	The change set is the reviewable step: the studio sees every resource action and replacement before anything is applied. A ROLLBACK_COMPLETE stack is deleted and waited out first, because CloudFormation refuses to update one.
GitHub Actions with GitHub OIDC	—	Where the build and the deploy actually happen; aws-actions/configure-aws-credentials@v4 assumes the stack's deploy role.	No cloud access key is ever written to GitHub on the AWS path. Both the classic subject and the immutable repo:owner@OWNER_ID/name@REPO_ID form are pinned in the trust policy, and a repository that has customised its OIDC subject template is refused up front.
GitHub CLI (gh)	—	Repository creation, identity lookup, Actions variables and secrets, workflow listing and cancellation, PR creation, and git authentication via gh auth git-credential.	The agent runs git without a terminal, so Git Credential Manager cannot prompt and the push fails with 'could not read Username' even while gh is signed in. The helper is scoped to the single command instead of mutating the user's global config the way gh auth setup-git would.
Docker, on the GitHub runner only	—	Builds the ECS image from a three-stage node:20-alpine Dockerfile inside the deploy workflow.	Explicitly not a local requirement — the ECS profile's required_tools is git, gh, aws. The Dockerfile header says it: built by GitHub Actions, so there is no need for Docker on the machine that generated it.
mongo:7 as a GitHub Actions service container	—	A throwaway database on 127.0.0.1:27017 for the duration of every build job, with a mongosh ping health check.	Making the build placeholder URI true is the fix that does not depend on predicting what the model wrote. The templates avoid connecting at import, but the model writes application code too — the failing app had mongodbAdapter(await getDb()) at top level in a lib/auth.js no template produced.

Technology	Version	What it does here	Why
nginx and systemd (EC2 target)	—	nginx listens on 80 and reverse-proxies to 127.0.0.1:PORT with forwarded headers and upgrade handling; a hardened nextjs.service unit runs node /opt/app/current/server.js with NoNewPrivileges, PrivateTmp, ProtectSystem=strict, ProtectHome and ReadWritePaths=/opt/app.	The application binds to loopback, so the only thing the security group's open port 80 reaches is nginx. Being same-origin behind that proxy is also why the Better Auth client patch strips its explicit baseURL.
Vercel CLI plus the Vercel REST API	—	The CLI does sign-in and vercel link; everything read back (deployments, build logs, domains) and every exact state change (promote, delete, env upsert) goes through REST at api.vercel.com.	vercel_api.py says it plainly: the CLI's subcommand names and JSON shapes have moved between versions, while these endpoints have not. The token is found by probing every location vercel login is known to write, including the Windows xdg.data layout under APPDATA.
PyYAML	>=6.0,<7	Parses the generated workflows and CloudFormation templates back, to verify they are structurally valid before a run may proceed.	A custom SafeLoader subclass drops the implicit bool resolver and accepts every ! tag — otherwise YAML 1.1 turns the workflow's on: key into the boolean True, and !Ref / !GetAtt / !Sub make a CloudFormation template unparseable.
jsonschema (Draft 2020-12)	>=4.23,<5	Validates the planner's JSON against PLAN_SCHEMA and BUILD_REPAIR_SCHEMA, with one repair round-trip that feeds the validation error back to the model.	The repair schema is an enum of four permitted actions, so the model can never propose an arbitrary file edit. The plan schema constrains instance types, ports (1024-65535) and the health path pattern.
Ollama via AgentForge's router (gemma4:31b-cloud by default)	—	Writes the deployment plan and picks build repairs, streamed over /api/chat with temperature 0 and num_predict 4096.	bridge.py: the copied client posted every model to localhost:11434, so :cloud tags only worked after a separate ollama signin. Routing through AgentForge sends cloud tags to ollama.com with the bearer token it already has. Native structured output is skipped entirely for :cloud models, falling back to validated JSON extraction.
pymongo[srv]	>=4.9,<5	The pre-deployment Atlas URI check: connect, ping, server_info, list database names.	The most expensive failure in this pipeline is a deployment that reaches AWS, passes its health check, and only then turns out to have an unusable database URI. This surfaces it in a couple of seconds.
ReportLab	>=4.2,<5	Renders the A4 evidence report PDF served from /api/runs/{id}/report.pdf.	—
requests	>=2.32,<3	Ollama streaming, the Vercel API, the live homepage and health probes, and AgentForge's proxy and job shim into the agent.	—
Playwright	>=1.48,<2 — documented, deliberately not installed under AgentForge	Screenshotted the standalone agent's dashboard tabs into the evidence bundle.	Root Requirements.txt says it is deliberately absent: it only drove the evidence exporter's dashboard screenshots, and that dashboard is not part of the port. capture_dashboard swallows the failure and returns an empty list, so both exports still work.

Technology	Version	What it does here	Why
Next.js 16, React 19, Tailwind 4 (the studio)	—	The Deploy tab: eleven client components under <code>studio/components/deploy/</code> , with <code>useMonitor</code> and <code>useRunData</code> hooks over a Zustand store.	Monitor polling is cadenced per target (Vercel 20s, EC2 35s, ECS 40s), pauses while the document is hidden, and only auto-polls while the run is <code>VALIDATING</code> or newly <code>LIVE</code> — a monitor call costs a round of live cloud API reads.

4. How it works

1. 1. Mount and recover

`server.py:start_deploy_api` puts `deployment-agent/` on `sys.path` and calls `deploy_agent.mount.serve`, which sets `DEPLOYMENT_AGENT_DATA_DIR` to `production-ready/.deploy`, runs `dfserver.start_reconciliation()` (one immediate `reconcile_once` sweep, then the 15-second supervisor loop) and blocks on `ThreadingHTTPServer` at `127.0.0.1:7834`. Nothing in `mount.py` may raise into `AgentForge`: a broken deployment agent must be a tab that explains itself, reported through `/api/deploy-status`.

2. 2. Analyze (DRAFT to ANALYZING)

`POST /api/runs/analyze` requires `cloud_consent` true, creates the run row and spawns `Orchestrator._analyze`. `IntakeAgent` copies the project into the staged worktree, finds every `package.json` with a next dependency (shallowest first), detects the package manager from the lockfile, generates a `package-lock.json` inside the copy if none exists, scans `process.env` references for the environment contract, enumerates App Router routes, and records the git branch, remote and dirty files.

3. 3. Plan

`PlannerAgent` fills the plan with deterministic values from discovery first, then asks the model for risks, recommendations, source patches and a generation block. Only three fields of the answer are actually used — the GitHub job list, the instance type and the requested patch names; everything unsafe (commands, root, port, health path) stays authoritative from discovery. `free_tier_only()` clamps a `t3.small` or `t3.medium` choice down to a micro. A failure here is non-fatal: `model_used` stays False, and the deploy gate later refuses such a run.

4. 4. Generate

`ArtifactGeneratorAgent.generate` resolves the environment contract, writes `.env.example`, applies the compatibility patches, writes `ci.yml` and the target's `deploy.yml`, then the target's own set — `infra/bootstrap.yml`, `infra/parameters.example.json` and `deploy/release.sh` for EC2; those plus `Dockerfile`, `.dockerignore` and `deploy/task-definition.json` for ECS; `deploy/vercel-environment.json` and (unless the user already has one) `vercel.json` for Vercel. It finishes with `deployment-report.md`, `readiness-score.json` and `deployment-manifest.json`. Each record carries its SHA-256, its size, and whether an original file existed and what its hash was.

5. 5. Validate and build (to REVIEW_READY)

`SecurityValidatorAgent` scans every artifact for secret-shaped values, parses every JSON and YAML file, checks the target's required artifacts are present, checks `deploy.yml` has contents: `read` plus `id-token: write` (and that Vercel does not request an `id-token`) and a `workflow_dispatch` trigger, and target-specifically checks the OIDC subject restriction, the ECS `iam:PassedToService` condition and that the `Dockerfile` copies `.next/static`. Then the orchestrator runs the real install and build. The gates `model_used`, `artifacts_valid`, `build_validation` and `security_validation` are recorded, and the run parks at `REVIEW_READY`.

6. 6. Approve (REVIEW_READY to BOOTSTRAPPING)

`POST /api/runs/{id}/deploy` needs `approved` true and a MongoDB URI. `_validate_request` re-checks every precondition and re-hashes every artifact; `_reserve_project` takes an in-memory lock on the resolved lowercase project path and also scans persisted runs for another in an active state, raising `DeploymentConflictError` (HTTP 409) either way. The worker thread then checks the target's required CLI tools are installed and transitions to `BOOTSTRAPPING`.

7. 7. GitHub identity and OIDC subjects

The repository is created private if there is no origin (with a timestamp suffix on a name collision), its real default branch is read back from the API, and two OIDC subjects are computed: the classic `repo:owner/name:ref:refs/heads/<branch>` and the immutable `repo:owner@OWNER_ID/name@REPO_ID` form. A repository whose OIDC subject template has `use_default` false is refused here rather than at `configure-aws-credentials` time.

8. 8. Prepare the provider

AWS: find a reusable account-level OIDC provider (never one this stack owns), pick a network (a dedicated VPC normally; two existing public subnets in different AZs when the region already holds five VPCs), create and execute the change set, wait on `stack_create_complete` or `stack_update_complete`, then put `_secret_value` with `MONGODB_URI`, a freshly minted `BETTER_AUTH_SECRET`, and `BETTER_AUTH_URL` and `BASE_URL` taken from the stack's `ApplicationUrl` output. Vercel: vercel link in the staged worktree, read `projectId` and `orgId` from `.vercel/project.json`, resolve the production URL, and write the production environment variables straight to Vercel. Each returns a `ProviderPrep` of GitHub variables, GitHub secrets and repo state.

9. 9. Hand GitHub what it needs

`_set_github_variables` writes each non-secret with `gh variable set --body` and raises 'Bootstrap output is missing X' on any empty value, which is why every one of those names must be a template Output. `_set_github_secrets` writes secrets with the value on stdin, never `--body`, so the credential never appears in `argv` where a process listing or an echoed `stderr` would expose it.

10. 10. Apply, commit, push (to CI_RUNNING)

Reviewed files are copied from the staged worktree into the real repository, with pre-existing originals backed up under `<run>/backup/` and any file that changed since review aborting the run. `git add -A` stages the application's own edits too — a redeploy that staged only the artifacts produced a commit of `deployment-manifest.json` and `deployment-report.md` and left the site serving the previous code. The push either lands on the default branch or falls back to a PR. If nothing was staged at all, `gh workflow run deploy.yml` is dispatched so a redeploy of unchanged code still deploys.

11. 11. Watch the workflow (CI_RUNNING to DEPLOYING to VALIDATING)

`_wait_for_workflow` polls `gh run list` for up to 900 seconds, selecting the run by `headSha`: `select_workflow_run` treats the reviewed commit SHA as authoritative, because a repository can hold older successful deploys of the same workflow and taking the newest run would let a stale success advance a fresh deployment. It emits a monitor event whenever the status or conclusion signature changes, raises on failure, cancelled, `timed_out` or `action_required`, and hands off to the dashboard if the workflow is still running when the budget expires.

12. 12. Validate live (VALIDATING to LIVE)

`MonitorAgent.snapshot` collects GitHub, the provider and the logs, probes `/` and `/api/health`, and scores build plus `cicd` 20, provider 25, security, monitoring 10 and api 10. It promotes to LIVE only when the state is VALIDATING or LIVE, the score is at least 90, and every probe passed. Because the ceiling is exactly 100 with no headroom, `AgentForge's _deploy_wait` adds an escape: after 240 seconds parked in VALIDATING it asks the site itself, and a URL that answers is the answer — skipping the production build alone zeroes a 15-point category and makes 90 unreachable for ever.

13. 13. Adopt, or retire

`adopt_deploy` copies the run, its events, the last monitor snapshot and a link record into the project's `.agentforge/deploy/`. On every later read of the Deploy tab, `_deploy_run_state` asks the agent for the run's current state rather than trusting that snapshot; if it now reads `DESTROYED`, `retire_deploy` moves the whole folder to `.agentforge/deploy-archive/<run_id>/` and drops a `deploy-deleted.json` marker — otherwise the tab kept drawing an instance and a stack for infrastructure that no longer existed.

5. The code, file by file

Path	What lives there
deployment-agent/deploy_agent/dfagents/generator.py	2,175 lines and the heart of the subsystem: both CI and deploy workflow generators per target, both CloudFormation templates, the Dockerfile, the task definition, the EC2 release script, the Vercel config and environment mapping, the compatibility patches, and the manifest, report and readiness writers.
deployment-agent/deploy_agent/dfagents/deployer.py	1,549 lines: pre-deploy verification, the project reservation lock, the three <code>_prepare_*</code> provider halves, the CloudFormation change-set apply, GitHub variable and secret delivery, apply-commit-push, workflow waiting, cancel, teardown, resource draining and rollback.
deployment-agent/deploy_agent/deployment_agent/state.py	StateStore: the SQLite schema, <code>ALLOWED_TRANSITIONS</code> , and <code>transition_run</code> , which raises <code>RuntimeError('Invalid deployment state transition: X -> Y')</code> on any edge not in the table.
deployment-agent/deploy_agent/dfs/erver.py	The HTTP API. A copy of the standalone <code>server.py</code> with four things removed — the <code>:7835</code> websocket, the static dashboard, the browser it opened at start, and the folder picker — and nothing added.
deployment-agent/deploy_agent/dfagents/monitor.py	The snapshot collector for GitHub, EC2, ECS and Vercel; <code>select_workflow_run</code> ; readiness scoring; the transitions to <code>DEPLOYING</code> , <code>VALIDATING</code> and <code>FAILED</code> ; and <code>_mask_identifiers</code> , which masks 12-digit account ids and long sha256 digests.
deployment-agent/deploy_agent/deployment_agent/orchestrator.py	The analysis pipeline — intake, plan, generate, validate, local build, one repair round, finalize — and the readiness bookkeeping that fills the gates dictionary the deployer later checks.
deployment-agent/deploy_agent/dfagents/planner.py	OllamaClient (streaming chat, native-schema probe with fallback, JSON extraction, one schema-repair round) and PlannerAgent with <code>PLAN_SCHEMA</code> , <code>BUILD_REPAIR_SCHEMA</code> and the free-tier instance clamp.
deployment-agent/deploy_agent/dfagents/validator.py	SecurityValidatorAgent: secret scan, JSON and YAML parse checks, required-artifact check, workflow permission checks, per-target template and Dockerfile checks, and the bool-resolver-free YAML loader.
deployment-agent/deploy_agent/dfagents/intake.py	Staging and discovery: Next.js service detection, package manager and lockfile, <code>process.env</code> scan, route scan, port detection from scripts, and the git repository spec.
deployment-agent/deploy_agent/deployment_agent/environment.py	EnvironmentContractResolver — the five resolutions, the provider-managed and auto-generated name sets, <code>DEPLOYER_INJECTED = (BETTER_AUTH_URL, BASE_URL)</code> , and the heuristic that marks a variable development-only when every one of its occurrences sits under a <code>NODE_ENV === development</code> guard.
deployment-agent/deploy_agent/deployment_agent/providers/spec.py	TargetProfile and the <code>TARGET_PROFILES</code> table — required tools, required artifacts, artifact kind, runtime strategy, whether OIDC permissions are needed, commit subject and PR title — plus <code>profile_for</code> , <code>target_of</code> and <code>ProviderPrep</code> .
deployment-agent/deploy_agent/deployment_agent/validation.py	The 19 <code>MANDATORY_GATE_IDS</code> , the <code>ValidationGate</code> helper, and <code>SemanticValidator</code> : the environment contract gate, the provider artifacts gate and the secret boundary gate.
deployment-agent/deploy_agent/deployment_agent/supervisor.py	<code>reconcile_once</code> and <code>start_supervisor</code> : the orphan test, the 20-minute and 90-minute grace periods, and the loop that keeps active deployments advancing without the dashboard being open.
deployment-agent/deploy_agent/deployment_agent/aws_onboarding.py	The SSO device flow manager, per-target IAM action and quota tables, <code>check_permissions</code> , <code>check_quotas</code> , <code>request_quota_increase</code> , <code>persist_sso_profile</code> and the bootstrap-role CloudFormation template.
deployment-agent/deploy_agent/deployment_agent/security.py	<code>SECRET_NAME_RE</code> , seven <code>SECRET_VALUE_PATTERNS</code> (AKIA keys, gh tokens, PEM private keys, mongodb URIs, URLs with inline credentials, bearer tokens), <code>redact_text</code> , <code>redact_data</code> , <code>json_dumps_safe</code> and <code>sha256_file</code> .

Path	What lives there
deployment-agent/deploy_agent/deployment_agent/credentials.py	InMemoryCredentialVault — a one-hour TTL store with no serialization surface, handing out opaque vault_ references and zeroing values on clear.
deployment-agent/deploy_agent/deployment_agent/mongodb_check.py	The Atlas URI preflight, including the list of query options the Node driver accepts and the auth, DNS and allowlist error classification.
deployment-agent/deploy_agent/deployment_agent/vercel_api.py	Thin REST helpers for projects, deployments, build events, env upsert and delete, promote, delete and domains — with the comment explaining why the env type is encrypted and not sensitive.
deployment-agent/deploy_agent/deployment_agent/vercel_auth.py	Probes every known vercel login token location, including the Windows xdg.data layout, and reports the connection identity for the dashboard without throwing.
deployment-agent/deploy_agent/deployment_agent/exporter.py	The evidence ZIP, the ReportLab PDF, the best-effort Playwright dashboard capture, and save_evidence for operator-pasted screenshots (PNG/JPEG/WebP data URLs, 12 MB cap).
deployment-agent/deploy_agent/bridge.py	The only AgentForge-aware module in the copy: the data directory, the settings reader, model routing for :cloud tags, the readiness probe and free_tier_only().
deployment-agent/deploy_agent/mount.py	Runs the agent inside AgentForge's process, and documents why it keeps its own threaded HTTP server instead of becoming ASGI routes.
server.py	AgentForge's side: start_deploy_api, _proxy_deploy, the deploy job shim, start_deployment, _deploy_autopilot, _deploy_wait, deploy_settings_summary, _deploy_mongo_uri, adopt_deploy, retire_deploy and read_deploy_results.
studio/components/deploy/DeployPanel.jsx	The Deploy tab shell: target picker, account readiness list, test-failure override, the monitor view bar and the ten panes.
studio/components/deploy/DeployDanger.jsx	Cancel and Delete, with two different confirmation dialogs that spell out exactly what each does and does not touch, and which sends force true when the run is in an active state.

6. Why it is built this way

Design decisions with their reasons. Where the source records the failure that forced a choice, that failure is given rather than a general principle.

Decision	Why
Every build job runs a real mongo:7 service container on 127.0.0.1:27017	next build's 'Collecting page data' pass imports and executes every page and route, so a module that touches the database at import opens a socket while the app is being compiled. On run 8dd72476 (pubudu-tire-pos) that produced 'Failed to collect configuration for /api/auth/[...all] [cause]: MongoServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017'. The route named in the error moved between /, /admin, /api/products/[id] and the auth route across runs, which is why reading the message led nowhere and only a reproduction found it. The templates were made lazy, but the model writes the app's own modules too — the failing app had mongodbAdapter(await getDb()) at top level in a lib/auth.js no template produced. Making the placeholder URI true is the fix that does not depend on predicting what the model wrote.
docker build --network=host on the ECS deploy workflow	The ECS target compiles inside docker build, and a build container has its own network namespace: the runner's MongoDB service is not reachable at 127.0.0.1:27017 from inside it. Without the flag any module that touches the database at import fails the image build — the same failure the service container exists to prevent. test_build_database.py::TheImageBuildCanReachIt pins it.
The Better Auth catch-all route is patched to build its handler on the first request	The generated route did export const { GET, POST } = toNextJsHandler(auth.handler) at module scope, so importing it constructed Better Auth, which connects to MongoDB. Adding a database to CI alone would have hidden it rather than fixed it, and no app should need its database up to compile. The patched module keeps a lazy handlers variable and does await import('@lib/auth') inside a ready() helper, with dynamic force-dynamic. It only rewrites the exact shape the builder emits: a hand-written route is untouched, and a second pass reports no patch, so the audit trail does not lie.
lib/mongodb.js is rewritten to connect on first use, with a thenable default export	A regex replaces the eager clientPromise block with a connection() function and exports { then: (ok, no) => connection().then(ok, no) }, and rewrites await clientPromise to await connection(). The thenable shape means every existing call site keeps working unchanged. The replacement's own comment says why: connecting at import made a running database a requirement to COMPILE, and CI has none.
The build environment and the runtime secret are each defined in exactly one place	Three build sites — the CI workflow, the EC2 deploy workflow and the Dockerfile's build stage — each spelled their own env block out by hand, so all three set MONGODB_URI and none set the Better Auth pair. On a real bccycle run the build logged '[better-auth] Base URL is not set' and 'You are using the default secret', then reported success in 46 seconds. That cost the one signal separating a broken deployment from a healthy one, because 'Base URL is not set' is the exact line the live Invalid origin failure prints. _build_environment and _runtime_secret_entries are now the single lists both ends are built from.
BETTER_AUTH_URL and BASE_URL are read from the stack outputs into the runtime secret	On a live deployment, signing up returned 'Invalid origin' with '[better-auth] Base URL is not set' in the container log, while the homepage answered, /api/health answered and readiness scored 100 — only a human trying to sign up found it. The URL is knowable here and nowhere earlier: it is the load balancer's DNS name or the Elastic IP, both of which exist only once the stack is applied. MONGODB_DB is deliberately not written, because an empty value would override the driver's fallback to the database named in the URI with 'test'.

Decision	Why
An OIDC provider the bootstrap stack itself owns is reported as absent	ExistingOidcProviderArn tells the template not to create one, which drops GitHubOidcProvider out of the stack. On a-neighbourhood-repair-cafe the first deploy created it (CREATE_COMPLETE 08:38); the second found it, passed it back, and CloudFormation deleted it (DELETE_COMPLETE 08:56) while the deploy role's trust policy still named that ARN. Every second deployment then failed at configure-aws-credentials with 'The web identity token provided could not be validated', a message that points nowhere near the cause. <code>_find_github_oidc</code> now lists the stack's own resources and returns empty if it manages one.
The ECS service is created with DesiredCount 0 and scaled to 1 by the workflow	CloudFormation waits for a load-balanced service to stabilise, and the placeholder busybox image can never pass a health check on the application's own health path. Measured on a real account: ECS killed and replaced the task on a loop ('replaced 1 tasks due to an unhealthy status') while the stack sat in CREATE_IN_PROGRESS and the load balancer billed by the hour. With zero desired tasks the service creates immediately and the first real rollout is what starts a container. DeploymentController Type ECS stops CloudFormation dragging the service back to the placeholder on the next update.
The load balancer and target group are left unnamed	Elastic Load Balancing caps those names at 32 characters and a project slug is already up to 31: 'a-neighbourhood-plant-nurser-16-tg' is 35 and failed the stack outright. CloudFormation generates a name from the stack and logical id that always fits.
The image is tagged with github.sha and never :latest, and ECR tags are immutable	Two things depend on it and both break silently without it: rollback has no earlier revision to name, and the readiness check cannot tell whether the running service came from this commit or an older one. MonitorAgent._readiness for ECS literally asserts the head SHA appears in the running task's image.
Teardown can be forced, and forcing it cancels the run first	Refusing outright was a dead end and a costly one. 'Wait for it to finish or fail' assumes it will do one or the other, and the monitor only stamped FAILED from CI_RUNNING and DEPLOYING — so anything that reached VALIDATING and then broke stayed VALIDATING. Run 3fc71a2466 sat there for over a day, its instance answering 502, still billing, with no way to delete it from the dashboard; the AWS console was the only way out, which is exactly the situation the button exists to prevent. The order matters: a teardown racing a live GitHub Actions run can delete a stack the workflow then recreates.
One shared definition of active (models.ACTIVE_STATES), used by the studio too	The studio computed running as !TERMINAL.has(state) && !error, so an active run that had recorded an error read as idle and the panel offered a Delete the agent was always going to refuse. DeployDanger now tests membership of the same four states, and test_stuck_run_teardown.py asserts <code>_PERSISTED_ACTIVE_STATES</code> equals <code>ACTIVE_STATES</code> and that every active state has an edge to FAILED.
Cancel exists as its own state, with edges to BOOTSTRAPPING and DESTROYED	Before this there was no CANCELLED state and no endpoint at all, so a run wedged in BOOTSTRAPPING or CI_RUNNING had no exit: teardown refused it, nothing could stop it, and the supervisor kept reconciling it every sweep. CANCELLED sits outside _RECONCILED_STATES so the supervisor leaves it alone, and its two outgoing edges are what make 'cancel, then either redeploy or delete' possible at all — transition_run is really enforced, so the sequence only works because both edges exist.
Vercel environment variables are written as encrypted, not sensitive; secrets reach GitHub over stdin; the instance size is clamped to the free tier	Vercel's sensitive variables are write-only: they reach the running function, but vercel pull returns the literal placeholder [SENSITIVE], so a Next.js build that reads the value fails with whatever error that placeholder produces; encrypted is still encrypted at rest but is decrypted for builds. Separately, gh secret set --body would put the credential in argv where a process listing or an echoed stderr exposes it, so run_command passes it on stdin. And the planning model chose t3.small unprompted on the first real AWS run — about 15 dollars a month, decided by a model, on an account whose owner had asked for the free option — so free_tier_only() defaults to True and clamps to a micro, keeping the architecture (t4g.small becomes t4g.micro) so the image is not rebuilt. It is a clamp, not an override: aws_free_tier false gives the plan its choice back.

7. Interfaces

Name	Shape
POST /api/runs/analyze	{cloud_consent: true (required), path, target: aws_ec2 aws_ecs vercel, validate_container} -> 202 {run_id, state: ANALYZING}
POST /api/runs/{id}/deploy	{approved: true (required), mongodb_uri, aws_profile, region, credential_reference, vercel_token} -> 202 {accepted, run_id}; 409 on a project conflict, 400 on any failed precondition
POST /api/runs/{id}/cancel	{confirm: true} -> 202 {cancelled, state: CANCELLED, workflow_cancelled: gh run id or empty}
POST /api/runs/{id}/teardown	{confirm: true, credential_reference, force} -> 202 {accepted, slug}; refuses an active run without force, refuses a DESTROYED run outright
POST /api/runs/{id}/rollback and /retry	rollback -> 202 {accepted} (SSM relink on EC2, promote on Vercel); retry -> 202 {run_id} for a fresh analysis of the same path and target
GET /api/runs/{id}/events?after=N	{events: [{event_id, run_id, type: step state log terminal prompt monitor change_set warning error, stage, status, percent, message, data, timestamp}]} — the replayable cursor that replaced the websocket
GET /api/runs/{id}/monitor	{captured_at, github{repository, runs, successful, failed, pull_request}, aws{region, stacks, instances or services and tasks, releases}, vercel{deployments, domains}, logs, api, errors, workflow, readiness{score, categories, phase}} — redacted and identifier-masked
GET /api/runs/{id}/artifacts, /artifact?path=, /diff	the ArtifactRecord list; a single file, but only if its path is in the persisted manifest and resolves inside the staged root; and the full unified diff of staged versus original
GET /api/runs/{id}/export.zip and /report.pdf	attachment downloads named deployment-evidence-<run8>.zip and deployment-report-<run8>.pdf
POST /api/aws/sso/{start,poll,roles,select}	start{start_url, region} -> {flow_id, verification_uri_complete, user_code, interval, expires_in}; poll -> {status: pending complete, accounts}; select{flow_id, account_id, role_name, persist_profile} -> {credential_reference, account_id, role_name, region, expires_at}
POST /api/aws/preflight, /quota/request, /bootstrap-role-template	preflight{target, credential_reference or aws_profile, region} -> {target, quotas[], permissions{status, principal, allowed, denied}}; the others request a quota increase and emit a grant template
POST /api/mongodb/check	{uri} -> {ok, code: connected malformed auth dns unreachable failed unavailable, message, server_version, database, database_in_uri, databases} — never the URI itself
GET /api/onboarding/status	{tools{git, gh, aws, node, vercel, ollama, ports}, aws_profiles, aws_identities{account, arn, region, service_ready, service_error}, github_authenticated, github_account, cloud_notice}
AgentForge proxy: /api/deploy/* and /api/deploy/jobs	/api/deploy/runs/x is proxied to /api/runs/x on 7834 with a 2-second connect and no read deadline; POST /api/deploy/jobs {path, method, body} -> {job_id: djob_...} polled at GET /api/deploy/jobs/{id} -> {status, http_status, result, error, elapsed}
AgentForge: /api/deploy-start, /api/deploy-results/{project}, /api/deploy-status	start{project, target, validate_container} kicks off autopilot; results -> {project, agent, live, have{last}, last{run_id, state, target, readiness, repo_state, error, link, monitor}, settings, deleted}; status -> {state, port, error, listening}
File contract: deployment-manifest.json	{version: 3, project, project_slug, primary_service, service_root, model, model_used, target, environment_contract, generation{...}, artifacts[], agent_owned_files[]} — the deploy gate rejects any other version and any set that does not match the persisted artifacts

Name	Shape
File contract: readiness-score.json	{score, categories{build 15, cicd 20, provider 25, security 20, monitoring 10, api 10}, phase: review deploying live, gates{model_used, artifacts_valid, build_validation, security_validation}}
File contract: deploy/task-definition.json placeholders	__IMAGE__, __EXECUTION_ROLE_ARN__, __TASK_ROLE_ARN__, __AWS_REGION__ and __RUNTIME_SECRET_ARN__:<NAME>:: — all substituted by the deploy workflow's render step, never by the agent
File contract: project .agentforge/deploy/	run.json, events.json, monitor.json, link.json{run_id, adopted_at, state, target}; on teardown the folder moves to .agentforge/deploy-archive/<run_id>/ and .agentforge/deploy-deleted.json{run_id, deleted_at, archive} is written
GitHub Actions variables and secrets the workflows read	EC2 vars: AWS_DEPLOY_ROLE_ARN, AWS_REGION, ARTIFACT_BUCKET, INSTANCE_ID, RUNTIME_SECRET_ID, PROJECT_SLUG. ECS adds ECR_REPOSITORY_URI, ECS_CLUSTER, ECS_SERVICE, ECS_TASK_FAMILY, ECS_EXECUTION_ROLE_ARN, ECS_TASK_ROLE_ARN, CONTAINER_NAME. Vercel vars: VERCEL_ORG_ID, VERCEL_PROJECT_ID, PROJECT_SLUG; secret: VERCEL_TOKEN